

CS3213 Foundations of S. Engineering (FSE)

AY24/25 Sem 2. github.com/gerteck

Software Engineering Foundations. Methodologies to develop working software, address customers' needs efficiently.

- Software development processes (agile vs. plan-driven)
- Requirements engineering, Modeling, Software architectures
- Software testing, Static analysis, Debugging, fault localization.

1. Software Engineering

Engineering discipline concerned with all aspects of software production, initial conception to operation and maintenance. Multiperson development of multiversion programs. People at core, code is important but only small part of building product.

- **Engineering:** selectively using (general and ambiguous) theories, methods, tools where appropriate.
- **Software engineering:** *practicalities* of developing, delivering useful software. Programming integrated over time. Software sustainability to keep software evolvable over time. (Time component)
- **People's** characteristics as a first-order success driver.
- **Origins:** 1965s software crisis, Margaret Hamilton, Apollo team director, coins software eng. Cost of bugs today, (2022 US \$2.41 trillion), technical debt biggest obstacle to changes to existing codebases, est. \$1.52 trillion. E.g. crowdstrike 2024 outage, DBS 2023, carousel data leak, Java Log4J 2021 Log4Shell zero-day vulnerability. Fred Brooks SWE – no silver bullet (Mythical Man Month).
- **Complexity:** *Essential complexity - inherent challenges*, e.g. communicating with stakeholders to determine product. *Accidental complexity* - can be solved, e.g. buffer overflow exploitation protection. Hardest single part – deciding what to build.
- **Techniques to address key challenges:** Address users' needs with software, correctness and reliable, efficiency with limited resources.

Central Software Engineering Activities

Software Specification: Define functionality, constraints on operation.

Software Development: Implement to meet specification.

Software Validation: Ensure performance & requirements met.

Software Evolution: Adapts the software to evolving customer needs.

“Shifting Left” - more costly to address a defect late in the process. SE processes have been “shifting left”.

SWE Qualities

- **Myths:** Genius Myth, 10x Dev. SWE is team activity. Working alone increases risks like design mistakes, irrelevance, and the “bus factor.”
- **Personal Principles:** Humility (Recognize limitations, self-improvement), Respect, Trust. Thrive in ambiguity, value feedback, challenge status quo, put users first, cares about team. Improving, Passionate, Knowledge about People, Organization, Sees Forest and Trees, Create shared context, Creates safe haven.
- **Software Product Attributes:** *Elegant:* Simple, intuitive designs that are easy to understand and maintain. *Anticipates Needs:* Build software that accommodates future requirements.

2. Software Engineering Processes

Software process models: Waterfall, incremental, integration & configuration. **Agile:** origins, scrum. Kanban. XP. **DevOps.**

Software Processes

Set of related activities leading to production of software system.

Software process model: simplified rep. of software process (e.g., sequence of activities). Illustrates different approaches to developing software.

- **Waterfall Model:** Simple, linear plan-driven software process model. Output of one phase feeds into the next. Development starts only after req. eng and design. **Pros/Use Cases:** Embedded-hardware /Safety-critical /Large systems. Works well if requirements clear, large projects, or distributed teams. **Cons:** accommodate change difficult.
- **Incremental Development:** Overlapping phases of specification, development, and validation. Built in increments, adding functionality. Can be plan-driven, agile, or mix. **Pros:** Lower cost of accommodating change, faster delivery. **Cons:** Harder to measure progress, may lead to system degradation (require refactoring).
- **Integration and Configuration:** Focuses on reusing existing components, frameworks, and libraries rather than build from scratch. AKA **reuse-based development**. Req. may need to be redefined based on identified components. **Pros:** Reduce amt. of software to develop, lower costs, risks. **Cons:** Limited customization, dependency on third-party, integration complexity.

Agile Software Engineering

Agile software development frameworks typically not complete software process models (e.g., Scrum is not specific to developing software). Designed for changing, unclear requirements and developing software in increments, delivering value quickly.

- **Agile:** Both mindset and concrete mngmt engineering framework. Focus on deliver value, avoid waste. Feedback loops, customer involvement. Adaptive to change. Lightweight process, empower people, lean manufacturing, buzzword certifications.
- **Historical Development and Origins:** 1980-90s: Traditional planning, QA, structured development processes. Late 1990s: Dissatisfaction, rise of agile alternative to rigid waterfall models. 2001: Agile Manifesto created by representatives from frameworks like Scrum, XP, Feature-Driven Development (FDD). Influenced by *lean manufacturing*, *The New New Product Development Game (1986)* – overlapping stages, build in instability subtle control, self-organizing teams (autonomy, self-transcendence of teams, cross-fertilization, learning & learning transfer cross teams).
- **Lean software development:** Limit waste (hardship in daily work) – partially done work, non-value adding processes/features, task-switching, waiting for input, defects, handoff motion.
- **Manifesto Principles:** Individuals & interactions over processes, tools. Working software over compreh. documentation, deliver frequently. Customer collaboration over contract negotiation (work tgt daily), Responding to change over following plan (welcome changing req.). face-to-face conversation, working software as progress measure, promote sustainable development. technically excellence and good design enhances agility, simplicity is essential. regular reflection.

- **Agile Adoption :** Larger the org, more likely to use hybrid mode. Bigger teams likely still use waterfall. Agile has concrete benefits - better collaboration, alignment to business needs, better quality software. **Adoption Problems:** business side slow to embrace. Too many mixed systems, siloed teams cause delays. Incorrect practice of agile, management buy-in, lack of documentation pain.

Scrum

Often synonymous to agile approaches. General, lightweight framework for project management.

- **Core Principles:** Empirical process control and lean thinking. Focuses on delivering value through increments in sprints.
- **Scrum Pillars:** Transparency (work and process visibility), Inspection (frequent evaluation), and Adaptation (adjustments as needed).
- **Scrum Team:** Self-managing, no hierarchies, typically ≤ 10 members. *Scrum Master:* Facilitates Scrum process, removes impediments. *Product Owner:* Maximizes product value, manages Product Backlog. *Developers:* Responsible for creating Increment each Sprint.
- **Scrum Events:** *Sprint:* Timeboxed iteration (≤ 1 month), deliver working Increment. *Sprint Planning:* Defines Sprint Goal, selects backlog items. *Daily Scrum:* 15-minute daily sync to track progress and adjust plan. *Sprint Review:* Team present results, adjust Product Backlog. *Sprint Retrospective:* Reflect on improvements for future Sprints.
- **Scrum Artifacts:** *Product Backlog:* Ordered list of work items aligned with the Product Goal. *Sprint Backlog:* Selected items for Sprint with an actionable plan. *Increment:* Deliverable output that meets the Definition of Done.
- **Timeboxing:** Sprint(S): ≤ 1 mnth. S. Planning: ≤ 8 hrs. Daily Scrum: 15 minutes. S. Review: ≤ 4 hrs. S. Retrospective: ≤ 3 hrs.
- **Kanban Board:** Often used in Scrum to visualize workflow. It helps teams track progress, limit work in progress (WIP), and improve flow (movement of potential value through a system).
- **Definition of Workflow (DoW):** A shared understanding of how work items flow through the system. Includes *Units of value* (work items or user stories), defined states (stages) work items flow through, policies for how items move through each state. WIP limit controls enforced.

Extreme Programming (XP)

Highly influential agile methodology (late 1990s). “extreme” approach to iterative development and best practices.

- New ver may be built several times/day, increments delivered every 2 weeks, All tests run successful for every accepted build.
- **Pair Programming:** One dev programming, other reviewing, focus on big picture. (driver & navigator). Both can think and interact at same level of interaction (e.g., same level of expertise). If one greater expertise likely to dominate interaction. Switch roles after a time period, and switch partners frequently. *Pros vs Cons:* Fewer bugs, better code understanding, peer learning, but lower cost / scheduling efficiency, personal clash.
- **Collective Ownership:** Everyone responsible for all code, anyone can change any code. Pair prog facilitates this, as everyone better understands larger system, contribute to different parts of code.

- **Test-driven development:** Unit tests written before code, test all potential fail cases. helps development, facilitates other practices of CI, refactoring etc.
- **Continuous Integration:** Dev team should always work on latest ver. Code integrated to main branch frequently. Facilitate using unit tests. (GH Actions, Jenkins).
- **Simple Design, Refactoring:** “simple is best”, refactoring as part of the lifecycle, keep it maintainable, easier to extend.
- Others: on-site customer for access system requirements, sustainable pace for net effect on code quality / long term productivity.

DevOps

For incremental development, challenge in deploying software quickly and reliably. Involves collaboration btw. dev (Scrum teams) and operations teams.

- **DORA (DevOps Research and Assessment):** Framework for measuring and improving DevOps performance through key metrics like deployment frequency, lead time for changes, change failure rates, mean time to recovery. *Observability and Monitoring* for measuring DevOps performance. Monitoring of running code and infrastructure provides insights to optimize the value stream.
- **Silos:** Traditionally, dev and ops teams separated, conflicting goals. Developers focus on delivering features quickly, while operations prioritize stability and uptime. DevOps bridges gap by fostering collaboration.
- **DevOps Overview:** Unified approach, emphasize automation, collaboration, and continuous delivery. Popularized around 2010, it aims to make deployments routine and predictable. *DevOps* vs. *Agile*: Agile focus on iterative development, deliver value, DevOps extend by automating deployment pipeline. Emphasis on CI, delivery, feedback loops. "Infinite loop" of dev and ops.
- **DevOps Principles:** *Flow*: Reduce batch sizes and handoffs (pass of work across depts, teams) to streamline delivery. *Feedback*: Implement fast feedback loops to detect and resolve issues early. *Continuous Learning*: Promote experimentation and learning, with mechanisms to share knowledge across teams. Need safety culture.
- **Learning from Incidents:** Encourages *blameless postmortems* to investigate failures, implement preventive measures. Fosters culture of experimentation and learning.
- **Value Stream Mapping:** Technique to visualize and analyze the flow of work, identifying inefficiencies and improving delivery processes. (E.g. develop unit test - code solution - PR reviews - QA - regression, performance test - deploy - smoke test - UAT test)
- **Continuous Integration and Deployment (CI/CD):** Automates the integration, testing, and deployment of code changes, enabling frequent and reliable releases (push to production). *Infrastructure as Code*: Treats infrastructure configuration like code, using version control, automation to manage servers and environments.

Software Requirements Engineering

- **SRS: Software Requirements Specification:** Central document: software requirements document (or software requirements specification or SRS) • An official statement of what the system

developers should implement • It may include both the user requirements for a system and a detailed specification of the system requirements • Output of a requirements engineering phase • Covered in the next lecture

•